



**Student Name(s) and**

**Number(s) as per student card (s): 1) Karnishwar R (20086868)**

**2) Anandha Krishnan Sushandan (20055947)**

**3) Sanu Mathew (20075856)**

**Programme : Advanced Databases**

**Lecturer Name:- Jagadeeswaran Thangaraj**

**Module Title : B9IS125\_2526\_TMD1S**

**Assignment Title : “ASK” Event Booking and Management Company**

**Assessment:- CA 1- Group Assignment**

**By submitting this assignment, we are confirming that:**

- This assignment is all our own work;
- Any sources used have been referenced;
- we have followed the Generative AI instructions/ scale set out in the Assignment Brief;
- we have read the College rules regarding academic integrity in the QAH Part B Section 3, and the Generative AI Guidelines, and understand that penalties will be applied accordingly if work is found not to be my/our own.
- we understand that all work uploaded is submitted via Original, whereby a text-matching report will show any similarities with other texts.

## Table of Content

| <u>SI NO</u> | <u>Table of Content</u>            | <u>Page Number</u> |
|--------------|------------------------------------|--------------------|
| 1.           | Business Case                      | 3                  |
| 2.           | Database Scope                     | 4                  |
| 3.           | Business Requirements              | 5                  |
| 4.           | ER Diagram                         | 6                  |
| 5.           | Business Rules with Implementation | 8                  |
| 6.           | 3NF Justification                  | 9                  |
| 7.           | Conclusion                         | 10                 |
| 8.           | Individual Contribution            | 11                 |
| 9.           | Bibliography                       | 12                 |

## 1. Business Case

- Event management is fast moving, customer focused industry which is becoming more and more popular day by day. Due to this the industry purely depends on coordination with one another like, if we take a theatre, the event management company and the theatre should have good coordination in order carry on services efficiently.
- Event management companies deal with many tasks at once like talking to customers, booking the specified venues for customers, managing distributors, assigning staffs, tracking payments and collecting feedback. The event management companies take the place of both customers as well as vendors.
- Nowadays some businesses still use outdated software, do manual paperwork instead of computerizing their work. This often leads to work being slow, confusions, many mistakes take place and sometimes important information is also not stored properly.
- For example let us take a wedding planner business, this business requires lot of information to be stored. If the information are not stored correctly then there will be issues like double booking same venue for different people at same time, losing client details, mixing of event dates, payment issues etc..
- These problems directly affect the company's reputation because nowadays even a small issue can make clients dissatisfied.
- To overcome these issues we have come up with Event management company called **"ASK Events"**.
- The aim of our project is to create a hybrid database, that is store information like clients names, events, dates, payment details in a normal SQL database so that the information stays organised and no duplication of data occurs and store changing information like customer feedback, photos etc.. in a databased like MongoDB as it efficiently stores frequently changing data.
- We have also planned to bring all event related activities into one application. This makes it easier for people to look for different events in a single place rather than going to different applications or sites.
- The system will also include different access levels for admins, employees, vendors, customers so that people can view only the information they are allowed access to.

- This system helps to increase workflow and allows better decision making. This also make it easier to generate reports like supplier performance, customer satisfaction, payment details etc..
- Generating reports also makes it easier to analyse booking trends like most booked places and vendors can focus more on the most booked place to even more improve the experience.
- Another benefit of our company is that it reduces manual work and avoids any clashes like overbooking during busy seasons.
- We are trying to create the application user friendly and efficient enough to take care entire event management process from booking to completion of event and getting feedback thus, improving customer satisfaction and company's reputation.
- In conclusion, we are trying to build an event management application which focuses on improving business efficiency and giving customers a stress free event search.

## **2. Database Scope**

### **Modules Included in our scope:**

- 1) Client Management :** Stores client information and client has their own id linked to their details.
- 2) Event Booking and Scheduling :** Stores details of venues and assigning correct venues to customers without double booking.
- 3) Employee Records :** Maintains details of all employees without duplication and manages employee payment details.
- 4) Vendor Management :** Manages all the available vendors, responsible for posting shop pictures and other details in the application making it visible to the customers, assigns vendors according to the customer's requirements.
- 5) Payment and Billing System :** Responsible for managing all payment related tasks. Keeping track of payment activities.

**6) Feedback System** : Responsible for getting and storing customer feedbacks, ratings etc..  
Uses MongoDB for managing changing results.

**Modules not Included in our scope:**

**1) Real-time Venue Availability** : System only manage for the places available inside the company's database. Place's available outside the company's database cannot be checked in real time.

**2) Payroll and HR Management** : Eventhough our system has employee information module, it only stores basic details like name, role, salary but HR tasks like employee attendance, performance reviews cannot be tracked.

**3) Advanced Finance** : Our system only stores simple payment information like payment status, received amount, pending amount. Advanced calculations like discounts, EMI etc.. cannot be calculated.

Eventhough these modules are not present in our scope these can be added when project is further developed.

**3. Business Requirements**

The following business requirements were identified for Event Booking and Management System during our analysis phase.

**1) Unique Client Identification** : Each client must be given a unique CLIENT\_ID to which their phone number and other details are linked so that it is easy tracking clients, Fixes duplication.

**2) Event Management Assignments** : This ensures that only one event is assigned to a manager, this helps managers easily plan and execute the events efficiently.

**3) Vendor Engagement and Tracking** : Vendors are linked to multiple events based on the service they provide. This maintains a many-to-many relationship.

**4) Event Scheduling and Validity** : Each event must have a specific date and venue. This prevents overlapping of events and venues.

**5) Payment and Billing :** All payments must be tracked accurately and payments must be only marked only with status like pending, completed, cancelled. This ensure no invalid payment records and prevent payment inconsistencies.

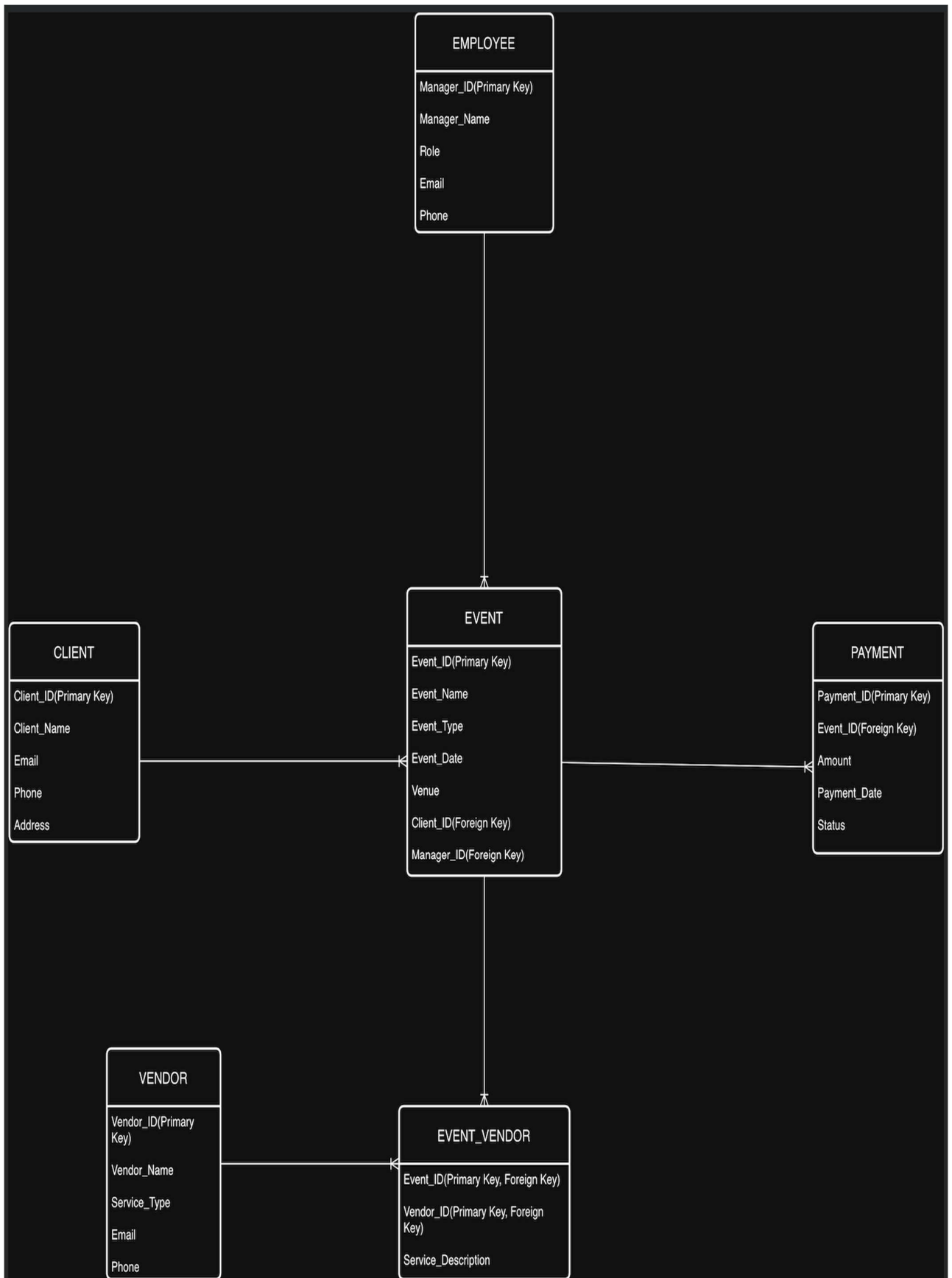
**6) Feedbacks and Media Support :** Clients should be able to give the feedback they want and they should also be able to upload videos, photos of events if they wish to. This is where MongoDB is helpful as it allows storing of inconsistent data.

**7) Timestamp Logging :** All bookings, payments, cancellations, vendor assignments must be timestamped as it is important for auditing.

#### **4. ER Diagram**

##### **ER Diagram for “ASK Event Booking and Management Company”**

- Below is the ERD for the Event management company.
- Single client can book multiple events so Client to Event is a one-many relationship.
- One employee can manage many events, so Employee to Event is a one-many relationship.
- Single Event may have multiple payment transactions, so Event to Payment is one-many relationship.
- Events can have multiple vendors, so Vendor are linked to Event with junction table, EVENT\_VENDOR.
- A single vendor can provide services many events. So, Vendor does to EVENT\_VENDOR is a one-many relationship.



### 5. Business Rules with Implementation

| Serial Number | Business Rule                                                                | Purpose                                                                | Implementation                                                                                                               |
|---------------|------------------------------------------------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 1             | Each client must be given a unique CLIENT_ID.                                | Prevents duplication of client information.                            | UNIQUE constraint on <i>CLIENT_ID</i> in CLIENT table.                                                                       |
| 2             | only one event is assigned to a manager.                                     | helps managers easily plan and execute the events efficiently.         | Link Manager_ID in EVENT table to Manager_ID in EMPLOYEE Table.                                                              |
| 3             | Vendors are linked to multiple events based on the service they provide.     | This maintains a many-to-many relationship.                            | Use a separate junction table called EVENT_VENDOR to connect events and vendors.                                             |
| 4             | Each event must have a specific date and venue.                              | prevents overlapping of events and venues.                             | Using a TRIGGER that automatically checks prebooking of the same venue before inserting a new event.                         |
| 5             | payments must be marked only with status like pending, completed, cancelled. | Ensure no invalid payment records and prevent payment inconsistencies. | Using CHECK constraint in the status column in PAYMENT Table to limit it to only 3 status.                                   |
| 6             | Clients should be able to give the feedback they want using MongoDB.         | allows storing of inconsistent data.                                   | Feedback is stored in <b>MongoDB</b> . Each Feedback entry includes Event_ID and Client_ID to link it back to relational DB. |



## **6. 3NF JUSTIFICATION**

Our Event Management system has been normalised up to Third Normal Form. Below, is the representation of how the 3NF table would look.

- **CLIENT (Client\_ID PK, Client\_Name, Email, Phone, Address)**
  - Client\_ID is primary key.
  - All non-key attributes fully depend on Client\_ID.
  - No transitive dependencies, every attribute only depend on Client\_ID.
  
- **EMPLOYEE (Manager\_ID PK, Manager\_Name, Role, Email, Phone)**
  - Manager\_ID is primary key.
  - All attributes depend on Manager\_ID.
  - No attribute depends on another non-attribute key.
  
- **EVENT (Event\_ID PK, Event\_Name, Event\_Type, Event\_Date, Venue, Client\_ID FK → CLIENT.Client\_ID, Manager\_ID FK → EMPLOYEE.Manager\_ID)**
  - Event\_ID is the primary key.
  - Attribute like type, date, venue depend on Event\_ID
  - Client\_ID and Manager\_ID are foreign keys and link to external entities.
  - No attribute depends on other key attributes.
  
- **VENDOR (Vendor\_ID PK, Vendor\_Name, Service\_Type, Email, Phone)**
  - No transitive dependencies.
  - Vendor information fully depends on Vendor\_ID.
  
- **EVENT\_VENDOR (Event\_ID FK → EVENT.Event\_ID, Vendor\_ID FK → VENDOR.Vendor\_ID, Service\_Description)**
  - The junction table provides a many-to-many relationship between events and vendors as mentioned.
  - Composite key Event\_ID and Vendor\_ID uniquely identifies each record.

- No additional dependencies between non-key attributes exist.
- **PAYMENT (Payment\_ID PK, Event\_ID FK → EVENT.Event\_ID, Amount, Payment\_Date, Status)**
  - Payment\_ID is the primary key.
  - Amount, Payment\_Date and status depend on Payment\_ID
  - No non-key attribute depends on another non-key attribute.
  - Event\_ID serves only as a Foreign Key and does not have any partial or transitive dependency.
  - This is a simple representation of how core relationships are covered :
    - Client → Event
    - Employee → Event
    - Vendor ↔ Event
    - Event → Payment

## 7. Conclusion

- The event Booking and management system makes it much easier for our event management company to handle day-to-day work in an organised and efficient way. Instead of dealing with scattered information and manual errors, the system brings clients, vendors, events, staff and payments into one structured place.
- The SQL database helps keep all important data clean and reliable. With the constraints like Primary Key, Foreign Key and Triggers the system automatically prevent issues such as Double Booking a venue or making the payment invalid or other physical errors. Since the database has been normalised to 3NF, It avoids confusion and keeps data consistent across whole system without duplication of data.

- To make the system, even more flexible and reliable, we added a NoSQL MongoDB database for storing feedback. This allows the company to save different type of comments, ratings and even media files which are inconsistent without having to change the normal database structure. As event company collect more feedback and forms. This approach is very useful.
- Overall, the system reduces manual work prevents common scheduling problems and improve coordination between clients and event managers and vendors. It also sets a strong foundation for future upgrades like advance reports, analytics and improved customer experience features.

## **8. Individual Contribution**

### **1) Karnishwar R (20086868) :**

I contributed mainly to the planning and design part of this project. I identify the main entities requirements for the event management system and helped in creating the ER diagrams. I also prepare the list of business requirements and help shape their business rules for their system. I worked on ensuring that the tables follow 3NF and check that there is no duplication in their design for the implementation phase. I will be responsible for creating the event and client tables and adding their foreign key relationships. I will also support the team during SQL testing and verification

### **2) Anandha Krishnan Sushandan (20055947) :**

My contribution was focused on the SQL coding part of the project. I created the SQL statements for the EMPLOYEE, VENDOR and PAYMENT tables. I added the cheque constraint for the payment status and ensured the correct data types were used in each field. I also supported the development of the trigger used for preventing venue double Booking. In the implementation stage, I will test the trigger and verify differential integrity and insert sample records to ensure that the table structures and constraints work as expected.

**3) Sanu Mathew (20075856) :**

I was responsible for preparing the documentation and database explanation in the report. I wrote the business case, scope and NoSQL feedback section and described how MongoDB is used for storing client feedback. During implementation, I will create sample MongoDB feedback documents and help demonstrate how the hybrid system handles both structured and structured data.

## **9. Bibliography**

- 1) Entity Relationship Diagram - ERD - Conceptual Model - Crow's Foot Notation by Good4College Video Class - <https://youtu.be/I-XHfuJ43ic?si=pMLyUzpAAKynn34z>
- 2) ReRelational Database Concepts: Foreign Keys and Referential Integrity by Michael Fudge – <https://youtu.be/1EjipPLyaQE?si=oUVI5h3M3TI6cOAL>
- 3) MongoDB in 5 Minutes with Eliot Horowitz by MongoDB - <https://youtu.be/EE8ZTQxa0AM?si=vMa03XwXfigZUfwH>
- 4) dbschema.com. “How to Design a Relational Database Schema (2025).”- <https://dbschema.com/blog/design/design-a-relational-database-schema/>
- 5) Draw.io Documentation - [Working with connectors](#) and [style connectors](#)
- 6) Lecturer’s PPT and Materials from Moodle.